

# Vì sao gọi SAP2000 (qua COM API) chậm hơn ~1.000 lần so với FEM tự viết trong bài toán tối ưu đa mục tiêu khung thép

Dự án: So sánh MOPSO / NSGA-II / PESA-II / MOWAA / MOFDA cho tối ưu khung thép 2 tầng 1 nhịp — 30/07/2026

## 1. Bối cảnh

Trong quá trình phát triển 2 bài toán so sánh thuật toán tối ưu đa mục tiêu cho khung thép, có 2 cách tính nội lực/chuyển vị khác nhau được dùng làm hàm mục tiêu:

- **Bài toán cũ** — dùng SAP2000 (phần mềm phân tích kết cấu thương mại) thông qua COM API để phân tích kết cấu thật, kiểm tra theo TCVN 5575:2024.
- **Bài toán mới** — dùng hàm FEM tự viết bằng MATLAB (TinhNoiLucKhung2T1N.m) cho đúng khung 2 tầng 1 nhịp cụ thể, đã được kiểm chứng khớp với SAP2000 (chuyển vị nút, nội lực từng thanh).

Khi so sánh thời gian chạy thực tế của 2 cách tiếp cận này, chênh lệch tốc độ là rất lớn — đây là bài học đáng ghi nhớ khi thiết kế bài toán tối ưu có vòng lặp đánh giá hàm mục tiêu số lượng lớn.

## 2. Số liệu so sánh thực tế

| Tiêu chí                         | Bài toán cũ (SAP2000 COM API)                         | Bài toán mới (FEM tự viết) |
|----------------------------------|-------------------------------------------------------|----------------------------|
| Ngân sách NFE / thuật toán       | 2.550 (MaxIt=50, PopSize=50)                          | 30.000                     |
| Thời gian / 1 lần chạy           | ~40-45 phút (có lúc ~83 phút do bug)                  | ~20-150 giây               |
| Thời gian / 1 lần đánh giá (NFE) | ~1,0 giây / eval                                      | ~0,001 - 0,005 giây / eval |
| Chênh lệch tốc độ                | Khoảng 1.000 - 1.500 lần cho mỗi lần gọi hàm mục tiêu |                            |

## 3. Nguyên nhân gốc rễ

### 1. COM/OLE là giao tiếp liên tiến trình (inter-process), không phải gọi hàm nội bộ

Mỗi lệnh SapObject.xxx(...) phải marshal dữ liệu qua ranh giới giữa 2 tiến trình khác nhau: MATLAB.exe và SAP2000.exe. Riêng phần overhead giao tiếp này đã tốn hàng chục đến hàng trăm mili-giây mỗi lệnh gọi. Ngược lại, code FEM tự viết chạy 100% trong cùng một tiến trình MATLAB, không tốn bất kỳ chi phí giao tiếp liên tiến trình nào.

### 2. Mỗi lần đánh giá trong SAP2000 = một chu trình phân tích kết cấu đầy đủ

Unlock model → gán lại tiết diện mới → dựng lại ma trận độ cứng toàn hệ → giải hệ phương trình → ghi kết quả vào database nội bộ (thường ghi ra file .sdb trên đĩa) → gọi API đọc lại từng joint/frame. Pipeline

này được thiết kế để kỹ sư bấm "Run" vài lần một ngày, không phải để một vòng lặp tối ưu gọi hàng chục nghìn lần liên tục.

### 3. Code FEM tự viết là hàm đóng (closed-form) cho đúng 1 bài toán cụ thể

Chỉ vài node, ma trận độ cứng nhỏ, giải trực tiếp trong RAM, không I/O ra đĩa, không tiến trình ngoài. Vì "biết trước" chính xác bài toán cần giải, nó bỏ qua toàn bộ lớp tổng quát hoá (mesh, đa loại phần tử, đa loại tải...) mà một phần mềm thương mại như SAP2000 buộc phải xử lý cho mọi trường hợp có thể.

### 4. Bug rò rỉ tiến trình SapStop.m làm vấn đề trầm trọng thêm

ApplicationExit() bị gọi thiếu tham số bắt buộc (bool FileSave) → COM ném lỗi ngay lập tức → nhảy vào catch → delete(SapObject) không bao giờ chạy → tiến trình SAP2000 "mồ côi" (ẩn, Visible=false) tồn tại sau mỗi lần chạy. Sau 8 lần chạy batch, có tới 4 nhóm tiến trình SAP2000 cùng lúc (1 thật + 3 zombie) làm RAM tích lũy dần, khiến mỗi lần chạy chậm dần (từ ~44 phút lên ~83 phút/lần). Đây là lỗi cộng dồn thêm vào overhead COM vốn đã cao, không phải nguyên nhân chính.

## 4. Bài học rút ra

---

- **"Cùng một bài toán kỹ thuật" không có nghĩa "cùng một chi phí tính toán"**. Engine phân tích phía sau (FEM tự viết vs. phần mềm thương mại qua COM) quyết định số NFE và số lần lặp thống kê có thể chạy được trong thời gian hợp lý. SAP2000 giới hạn ở NFE=2.550, N=10 lần chạy (~22 giờ); FEM tự viết cho phép NFE=30.000, N=10 lần chạy chỉ trong vài phút.
- **Phần mềm thương mại qua API luôn có "thuế COM" không thể loại bỏ hoàn toàn**. Dù tối ưu code MATLAB đến đâu, phần overhead giao tiếp liên tiến trình vẫn tồn tại vì bản chất thiết kế của phần mềm đó không hướng tới việc bị gọi lặp lại hàng vạn lần.
- **Một FEM tự viết đã được kiểm chứng kỹ với phần mềm thương mại chính là một surrogate model đáng tin cậy**. Việc đã đối chiếu khớp chuyển vị nút và nội lực từng thanh với SAP2000 biến FEM tự viết thành công cụ tìm kiếm nhanh đáng tin cậy trong không gian thiết kế.
- **Bug hạ tầng (zombie process) có thể âm thầm nhân đôi thời gian chạy mà không liên quan gì đến thuật toán tối ưu**. Luôn kiểm tra Task Manager / log cảnh báo khi thấy thời gian chạy tăng dần bất thường qua từng lần lặp.

## 5. Lời khuyên áp dụng

---

### Cho bài báo khoa học

- Nêu rõ trong phần Methodology rằng code FEM tự viết đã được kiểm chứng (validate) từng chuyển vị nút và nội lực từng thanh so với SAP2000 trước khi dùng làm hàm mục tiêu.
- Nhấn mạnh đây là điểm mạnh về phương pháp luận: cho phép chạy khối lượng thống kê lớn hơn nhiều (NFE cao hơn, nhiều lần chạy độc lập hơn) trong cùng thời gian nghiên cứu — không phải "đi tắt" làm giảm độ tin cậy.
- Không so sánh trực tiếp thời gian tính bằng giây giữa 2 bài toán (SAP2000 vs FEM) — vì chênh lệch phản ánh chi phí engine phân tích, không phản ánh bản chất thuật toán. Nên so sánh ở mức "chất lượng nghiệm tại cùng NFE" (Hypervolume, Spacing, GD, IGD...).

### Cho bài toán vẫn dùng SAP2000

- Dù đã fix bug `ApplicationExit(false)`, không kỳ vọng tốc độ ngang FEM tự viết — mức ~40-45 phút/lần chạy (2.550 NFE) là "sàn" thực tế của COM API, không phải bug.
- Có thể cân nhắc thêm: gộp nhiều load case/lần phân tích trong 1 phiên SAP2000 thay vì mở/đóng liên tục; kiểm tra và tắt auto-save file .sdb mỗi lần eval nếu có, để giảm I/O đĩa.

## Nguyên tắc chung khi thiết kế bài toán tối ưu có vòng lặp lớn

- Dùng mô hình nhanh, đã kiểm chứng (surrogate) cho pha tìm kiếm (hàng chục nghìn lần đánh giá).
- Chỉ dùng phần mềm "chuẩn"/thương mại để verify lại một số ít nghiệm cuối cùng (ví dụ knee point của mặt trận Pareto), không dùng cho toàn bộ vòng lặp tối ưu.
- Luôn theo dõi thời gian chạy qua từng lần lặp — thời gian tăng dần bất thường là dấu hiệu rò rỉ tài nguyên (process/memory), không phải đặc tính tự nhiên của thuật toán.

---

*Nguồn: tổng hợp từ dự án MATLAB "MOPSO - NSGA2 - PESA2" và ghi chú bối cảnh TomTat\_BoiCanh.md của dự án song song dùng SAP2000 COM API.*